DAA HOME WORK ASSIGNMENT 03

submission date :10/03/2026
both in c and python

QUESTION 1: Breadth First Search Traversal

Problem Statement
Perform Breadth First Search (BFS) traversal on a given graph starting from a specified vertex.

Input

Graph (Adjacency List):

0 → [1, 2]

1 → [3]

2 → [3]

3 → []

Start Vertex = 0

Output

BFS Traversal = 0 1 2 3

Explanation
BFS visits vertices level by level using a queue.

QUESTION 2: Depth First Search Traversal

Problem Statement
Perform Depth First Search (DFS) traversal on the given graph starting from a specified vertex.

Input

Graph:

0 → [1, 2]

1 → [3]

2 → []

3 → []

Start Vertex = 0

Output

DFS Traversal = 0 1 3 2

Explanation
DFS explores one branch fully before backtracking.

QUESTION 3: Detect Cycle Using DFS

Problem Statement
Determine whether a cycle exists in the given directed graph using DFS.

Input

Graph:

0 → [1]

1 → [2]

2 → [0]

Output

Cycle Detected

Explanation
DFS detects back edges indicating a cycle.

QUESTION 4: Count Connected Components Using DFS

Problem Statement
Count the number of connected components in an undirected graph.

Input

Graph:

0 → [1]

1 → [0]

2 → [3]

3 → [2]

Output

Connected Components = 2

Explanation
Each DFS call marks one connected component.

QUESTION 5: BFS Shortest Path in Unweighted Graph

Problem Statement
Find the shortest path distance from the source vertex to all other vertices using BFS.

Input

Graph:

0 → [1, 2]

1 → [3]

2 → [3]

Source = 0

Output

Distances = [0, 1, 1, 2]

Explanation
BFS guarantees shortest paths in unweighted graphs.

QUESTION 6: Check if Graph is Biconnected

Problem Statement
Determine whether the given undirected graph is biconnected.

Input

Graph:

0 — 1 — 2

| |

4 ——— 3

Output

Graph is Biconnected

Explanation
A biconnected graph has no articulation points.

QUESTION 7: Find Articulation Points

Problem Statement
Identify all articulation points in the given graph.

Input

Graph:

0 — 1 — 2

  |

  3

Output

Articulation Points = [1]

Explanation
Removing vertex 1 disconnects the graph.

QUESTION 8: NP-Complete Problem Identification

Problem Statement

Determine whether the given problem is NP-Complete.

Input

Problem: 3-SAT

Output

Problem Type = NP-Complete

Explanation

NP-Complete problems are both in NP and NP-hard.

QUESTION 9: NP-Hard Problem Identification

Problem Statement

Classify the given problem based on its computational hardness.

Input

Problem: Traveling Salesman Problem (Optimization Version)

Output

Problem Type = NP-Hard

Explanation

NP-Hard problems are at least as hard as NP-Complete problems.

QUESTION 10: Difference Between NP-Complete and NP-Hard

Problem Statement

Identify the appropriate classification for the given problem.

Input

Problem: Decision version of TSP

Output

Problem Type = NP-Complete

Explanation

Decision versions of optimization problems often belong to NP-Complete.

QUESTION 11: Cook's Theorem Application

Problem Statement

Apply Cook's Theorem to classify the given problem.

Input

Problem: Boolean Satisfiability (SAT)

Output

SAT is NP-Complete

Explanation

Cook's Theorem proves SAT is NP-Complete.

QUESTION 12: Reduction Technique Identification

Problem Statement

Determine whether the reduction used is polynomial-time.

Input

Reduction: 3-SAT → Clique

Output

Reduction Type = Polynomial-Time Reduction

Explanation

Polynomial-time reductions preserve problem hardness.

QUESTION 13: Verifying NP-Completeness Using Reduction

Problem Statement

Verify NP-Completeness of a problem using reduction.

Input

Known NP-Complete Problem: SAT

Target Problem: Vertex Cover

Output

Vertex Cover is NP-Complete

Explanation

Reduction from SAT proves NP-Completeness.

QUESTION 14: Standard NP-Complete Problem Recognition

Problem Statement

Classify the given problem.

Input

Problem: Subset Sum

Output

Problem Type = NP-Complete

Explanation

Subset Sum is a classical NP-Complete problem.

QUESTION 15: Decision vs Optimization Problem

Problem Statement

Identify the category of the given problem.

Input

Problem: Is there a subset with sum = K?

Output

Problem Type = Decision Problem

Explanation

Decision problems have yes/no answers.

QUESTION 16: Polynomial-Time Verifier Construction

Problem Statement

Determine whether a polynomial-time verifier exists.

Input

Problem: Clique

Output

Polynomial-Time Verifier Exists

Explanation

Clique solutions can be verified efficiently.

QUESTION 17: Topological Sort Using Kahn's Algorithm

Problem Statement
Perform topological sorting using BFS (Kahn's algorithm).

Input

Graph:

A → B → C

Output

Topological Order = A B C

Explanation
Nodes with zero in-degree are processed first.

QUESTION 18: Check if Graph is DAG

Problem Statement
Determine whether a graph is a Directed Acyclic Graph.

Input

Graph:

0 → 1 → 2 → 0

Output

Graph is Not a DAG

Explanation
Cycles invalidate DAG property.

QUESTION 19: 0/1 Knapsack Problem

Problem Statement
Find the maximum achievable value using the 0/1 Knapsack approach.

Input

Weights = [2,3,4]

Values = [40,50,60]

Capacity = 5

Output

Maximum Value = 90

Explanation
Dynamic Programming ensures optimal selection.

QUESTION 20: Fractional Knapsack Problem

Problem Statement
Compute the maximum profit using fractional knapsack.

Input

Weights = [10,20,30]

Values = [60,100,120]

Capacity = 50

Output

Maximum Value = 240

Explanation
Items are chosen based on value-to-weight ratio.